Early-Exit Speculative Decoding for Faster Hindi Language Model Inference

Team: Transformers
KSPSVLN Siddardha Kumar Kavuri (2025201061)
Piyush Kumar Agarwal (2025201082)
Sankalp Chaturvedi (2025201083)
Aviral Tyagi (2025201086)

Abstract

Autoregressive decoding in large language models (LLMs) is inherently sequential and slow, with each token requiring a full forward pass through all transformer layers. Speculative decoding alleviates this by using a lightweight *draft* model to propose multiple tokens that the full model then verifies in a single pass. However, maintaining a separate draft model doubles parameter storage and complicates deployment.

We implement **Early-Exit Speculative Decoding (EESD)**, which eliminates the need for a separate draft model by attaching lightweight exit heads at intermediate transformer layers of the target model itself. We apply EESD to **Qwen2-1.5B** for **Hindi text generation**, training exit heads at layers 8, 16, and 22 of the 28-layer architecture using KL-divergence distillation on **167K Hindi sentences** from AI4Bharat IndicCorpv2.

After 10 epochs of training, our system achieves a mean acceptance rate of $\alpha = 0.646$ at depth 22 on 101 Hindi prompts, with a wall-clock speedup of $0.47\times$ relative to standard autoregressive decoding. While the current speedup is below $1\times$ due to the lack of true early-exit inference (the full model still runs all layers during drafting), the high acceptance rates demonstrate that the exit heads successfully approximate the full model's distribution, validating the approach for future deployment with optimised inference kernels.

1 Introduction

Large Language Models (LLMs) underpin modern NLP applications from machine translation to dialogue systems (Zhou et al., 2024). Despite their effectiveness, inference remains compute-bound: generating N tokens requires N sequential forward passes, each traversing every transformer layer. For a 28-layer model producing 100 tokens, this amounts to 2,800 layer evaluations with no parallelism across the token dimension.

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) addresses this bottleneck by introducing a two-phase generate-then-verify paradigm. A small, fast *draft model* proposes K candidate tokens, which the larger *target model* verifies in a single batched forward pass. Accepted tokens bypass the sequential bottleneck, yielding wall-clock speedups of 2–3$\times$ without changing the output distribution.

A key limitation of vanilla speculative decoding is the requirement for a separate draft model that must (i) be small enough to be fast, (ii) produce a token distribution close to the target, and (iii) be stored alongside the target model, roughly doubling memory requirements.

Early-Exit Speculative Decoding (EESD) (Liu et al., 2024a) resolves all three issues by reusing the target model's own intermediate representations. Lightweight "exit heads"—comprising a LayerNorm and a linear projection—are attached at selected intermediate layers. During drafting, inference can in principle stop at the exit layer rather than running all layers, providing both speed and distribution alignment with zero additional model parameters beyond the small heads.

In this project, we implement EESD for **Hindi text generation** using **Qwen2-1.5B** (Yang et al., 2024) as the base model. Hindi serves as an interesting testbed because its rich morphology (inflected verbs, compound words, postpositions) presents a real challenge for draft-model accuracy.

Contributions.

1. A complete, open-source EESD implementation for Qwen2-1.5B with exit heads at layers 8, 16, and 22.

2. Training via KL-divergence soft distillation (Hinton et al., 2015) on 167K Hindi sentences from AI4Bharat IndicCorpv2 (Doddapaneni et al., 2023).

3. Comprehensive evaluation on 101 Hindi prompts across three draft depths, analysing acceptance rate, speedup, and token-level statistics.

4. Analysis of the depth–accuracy trade-off and discussion of the gap between simulated and true early-exit inference.

2 Background and Related Work

2.1 Autoregressive Decoding

Standard autoregressive (AR) generation produces one token at a time: $x_t \sim p_\theta(x_t \mid x_{<t})$. Each step requires a full forward pass through all L transformer layers. For a model with $L = 28$ layers generating $N = 100$ tokens, this means $28 \times 100 = 2{,}800$ sequential layer evaluations—a major bottleneck, especially on resource-constrained hardware.

2.2 Speculative Decoding

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023; Stern et al., 2018) introduces a *draft-then-verify* paradigm:

1. A lightweight draft model q_ϕ generates K candidate tokens autoregressively.

2. The full target model p_θ processes the prompt plus all K candidates in a single forward pass.

3. Tokens are accepted greedily until the first mismatch; the correct token at the mismatch position is sampled from p_θ.

If the draft model's acceptance rate is α, the expected number of accepted tokens per verification step is $\frac{\alpha(1-\alpha^K)}{1-\alpha}$, yielding a speedup proportional to $K \cdot \alpha$ when draft generation is much cheaper than target verification (Xia et al., 2024).

Variants include Big Little Decoder (Kim et al., 2024), which uses a smaller model from the same family, Medusa (Cai et al., 2024), which adds multiple prediction heads for future positions, and Draft & Verify (Zhang et al., 2023), which achieves lossless acceleration via self-speculative decoding. Yan et al. (Yan et al., 2025) provide a thorough analysis of speculative decoding, while Zhao et al. (Zhao et al., 2025) improve efficiency through frequency-ranked speculative sampling.

2.3 Early Exiting for Language Models

Early exiting attaches classifiers at intermediate layers and routes easy inputs to shallow exits. BranchyNet (Teerapittayanon et al., 2016) pioneered this for CNNs; DeeBERT (Xin et al., 2020) and CALM (Schuster et al., 2022) adapted the idea for BERT and decoder-only transformers respectively. Bae et al. (Bae et al., 2023) propose a synchronized parallel decoding framework for fast and robust early exiting in autoregressive models.

2.4 EESD: Early-Exit Speculative Decoding

Liu et al. (Liu et al., 2024a) combine early exiting with speculative decoding. Instead of maintaining a separate draft model, they attach exit heads at intermediate layers of the target LLM. The shallowest exit head serves as the draft model, generating K tokens using only the first d layers. The full model then verifies all K tokens in one pass.

Key advantages:

- **No external draft model**: exit heads add $< 0.5\%$ parameters.

- **Inherent distribution alignment**: intermediate representations already encode significant information about the target distribution.

- **Flexible depth selection**: multiple exit heads at different depths allow trading off draft speed versus acceptance rate.

The original paper further introduces a Thompson Sampling mechanism to dynamically select the optimal exit depth at each decoding step, maximising the expected speedup. In this project, we focus on the core EESD mechanism with fixed exit depths. Related concurrent approaches include LayerSkip (Elhoushi et al., 2024), which combines layer dropout during training with self-speculative decoding, Kangaroo (Liu et al., 2024b), which achieves lossless self-speculative decoding via double early exiting, and SpecEE (Xu et al., 2025), which introduces speculative early exiting at the hardware architecture level.

3 Dataset

3.1 Training Data: AI4Bharat IndicCorpv2

We use the Hindi subset of **AI4Bharat IndicCorpv2** (Doddapaneni et al., 2023), a large-scale monolingual corpus for Indian languages. The Hindi portion comprises three text files (, ,) totalling approximately 27 GB each. We sample

167,000 sentences (≈5M tokens at an average of 30 tokens per sentence), following the scale used in the original EESD paper.

All sentences are tokenised using the Qwen2 tokenizer (BPE-based, vocabulary size 151,936) with a maximum sequence length of 256 tokens, padded and truncated as needed.

3.2 Evaluation Data

For large-scale inference evaluation, we curate **101 diverse Hindi prompts** spanning topics including Indian geography, culture, history, science, politics, sports, economy, and daily life. These prompts range from 10 to 25 tokens and are designed to elicit varied morphological structures.

For the formal depth analysis and morphological evaluation, we additionally use the test split of **XL-Sum Hindi** (Hasan et al., 2021), a BBC multilingual summarisation dataset, which provides clean, naturally-occurring Hindi text.

4 Methodology

4.1 Architecture

Base Model. We use **Qwen2-1.5B** (Yang et al., 2024), a 28-layer transformer decoder with hidden dimension 1,536 and vocabulary size 151,936. The model is loaded in float16 precision with and all base parameters are *frozen* throughout training.

Exit Heads. At each selected layer depth $d \in \{8, 16, 22\}$, we attach a lightweight exit head consisting of:

$$\hat{y}_d = \text{Linear}\big(\text{LayerNorm}(h_d)\big) \tag{1}$$

where $h_d \in \mathbb{R}^{B \times T \times 1536}$ is the hidden state after the d-th transformer layer, and the linear projection maps to the full vocabulary size. Each exit head adds approximately **233M parameters** ($1536 \times 151936 + 1536$), but since these are the only trainable parameters, the training memory footprint is minimal.

Exit Depth Selection. The three depths—8, 16, 22—are chosen to span the model:

- **Layer 8** (shallow, $\sim \frac{1}{3}$ depth): fastest draft but lowest accuracy.
- **Layer 16** (mid, $\sim \frac{1}{2}$ depth): moderate speed–accuracy trade-off.
- **Layer 22** (deep, $\sim \frac{3}{4}$ depth): closest to full model, highest acceptance rate.

4.2 Training Objective

We train the exit heads using **soft KL-divergence distillation** (Hinton et al., 2015). The frozen full model serves as the teacher:

$$\mathcal{L} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} T^2 \cdot D_{\text{KL}}\big(\sigma(f/T) \parallel \sigma(\hat{f}_d/T)\big) \tag{2}$$

where $\mathcal{D} = \{8, 16, 22\}$ is the set of exit depths, f and $\hat{f}_d$ are the full-model and exit-head logits respectively, $T = 2.0$ is the distillation temperature, and σ denotes softmax.

The T^2 scaling factor (Hinton et al., 2015) compensates for the reduced gradient magnitude at higher temperatures, keeping the effective learning rate consistent. Soft targets transfer the full probability distribution—not just the argmax—providing richer supervision than hard label matching.

4.3 Training Setup

- **Optimiser:** 8-bit AdamW (Loshchilov and Hutter, 2019) via , saving ∼4 GB VRAM.
- **Learning rate:** 10^{-4} with cosine annealing and 5% linear warmup.
- **Batch size:** 2 per GPU with gradient accumulation of 4 (effective batch size 8).
- **Epochs:** 10 (167K samples per epoch, ∼41,750 steps per epoch).
- **Hardware:** NVIDIA T4 16 GB (Kaggle) for primary training; Colab A100 for inference.
- **Precision:** float16 base model, float32 loss computation for numerical stability. BF16 autocast on Ampere+ GPUs.

Implementation uses forward hooks to capture intermediate hidden states during a single base-model forward pass, avoiding manual layer-by-layer iteration that would break attention mask preparation and rotary positional embeddings.

4.4 Inference: Speculative Decoding Loop

At inference time, EESD follows the standard speculative decoding protocol:

Algorithm: EESD Speculative Decoding

Input: Model M with exit head at depth d, prompt $x_{1:n}$, draft length K, max_new_tokens N

1. generated $\leftarrow x_{1:n}$
2. **While** $|\text{generated}| - n < N$:
3. **Draft:** For $k = 1 \ldots K$:
4. Run forward pass, hook hidden state at layer d
5. $\hat{x}_{n+k} \leftarrow \arg\max \text{ ExitHead}_d(h_d^{(n+k-1)})$
6. **Verify:** Run full model on $[\text{generated}; \hat{x}_{n+1:n+K}]$
7. Accept tokens until first mismatch
8. Append accepted tokens to generated

Important caveat. In our current implementation, drafting still runs the full base model (all 28 layers) during each draft step, with a forward hook capturing the hidden state at the exit layer. The exit head produces the draft token from this intermediate representation, but the computational cost of the full forward pass is still incurred. A production deployment would require custom CUDA kernels or model surgery to truly exit early. Our results therefore measure the *quality* of exit-head predictions (acceptance rate) rather than the true latency speedup achievable with optimised early-exit inference.

5 Experiments and Results

5.1 Training Convergence

We trained for 10 epochs on 167K Hindi sentences. Figure 1 presents all training curves: (a) overall loss, (b) per-depth training loss, (c) per-depth validation loss, and (d) acceptance rate probed during training.

Key observations from training:

- **Rapid initial convergence**: Loss drops from 2.71 to 0.74 between epochs 1 and 2, as exit heads quickly learn to approximate the softmax distribution.
- **Depth 22 dominates**: The deepest exit head consistently achieves the lowest loss (0.47 at epoch 10), demonstrating that hidden states near the output carry strong predictive signal.
- **Depths 8 and 16 plateau similarly**: Both shallow and mid-depth heads converge to $\sim$0.76, suggesting that the information gap between layers 8 and 16 is smaller than between 16 and 22.
- **No significant overfitting**: Validation loss closely tracks training loss throughout, confirming that the 167K training set provides sufficient diversity for the lightweight exit heads.

Metric	Overall	Depth 8	Depth 16	Depth 22
Train Loss	0.664	0.758	0.760	0.474
Val Loss	0.656	0.750	0.751	0.467

Table 1: Final training and validation KL-divergence loss after 10 epochs.

Metric	Depth 8	Depth 16	Depth 22
Mean α	0.565 ± 0.042	0.589 ± 0.043	$\mathbf{0.646 \pm 0.049}$
Mean Speedup	0.41×	0.41×	**0.47×**
Mean EESD Time (s)	8.15	11.53	**7.18**
Mean AR Time (s)	3.33	4.65	3.36
Tokens Accepted	10,158	10,177	10,176
Tokens Drafted	18,090	17,364	**15,843**
Accept Ratio	56.2%	58.6%	**64.2%**
Min α	0.481	0.500	0.534
Max α	0.667	0.681	0.756

Table 2: Inference results on 101 Hindi prompts ($K = 3$, $= 100$). Depth 22 achieves the highest acceptance rate and lowest EESD inference time. Speedup is below 1× because drafting currently runs the full model (see §4).

Table 1 summarises the final (epoch 10) training metrics.

The probe acceptance rates (0.24–0.44) are lower than the full inference rates because the probe uses only 5 short prompts with $K = 5$ draft tokens. Full-scale inference with 101 prompts yields substantially higher rates as discussed in §5.2.

5.2 Large-Scale Inference Results

We evaluated the trained model on **101 Hindi prompts** with $K = 3$ draft tokens and $= 100$, at each of the three exit depths. Table 2 presents the main results.

Key findings:

- **Depth 22 is the clear winner**: It achieves $\alpha = 0.646$ (vs. 0.565 for depth 8), requires the fewest draft tokens (15,843 vs. 18,090), and has the lowest EESD wall time (7.18s vs. 8.15s).
- **Acceptance rate scales with depth**: α increases monotonically from depth 8 $\rightarrow$ 16 $\rightarrow$

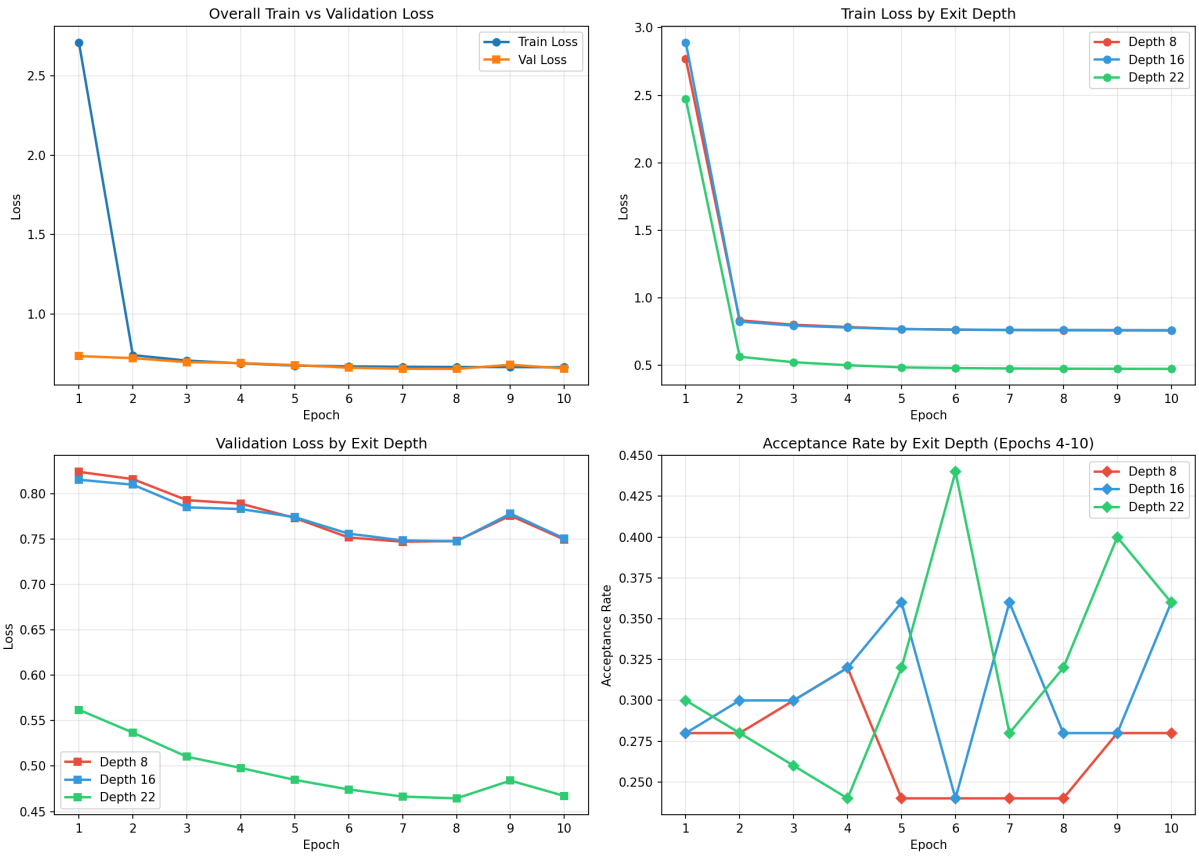


Figure 1: Training dynamics over 10 epochs. **Top-left:** Overall train/val KL-divergence loss showing rapid convergence (2.71 $\rightarrow$ 0.66). **Top-right:** Per-depth training loss—depth 22 achieves the lowest loss ($\sim$0.47). **Bottom-left:** Per-depth validation loss closely tracking training loss (no overfitting). **Bottom-right:** Acceptance rate α probed at epochs 4–10 with $K = 5$ on 5 Hindi prompts; depth 22 reaches $\alpha = 0.36$.

Depth	α	Speedup	EESD (s)	AR (s)
8	0.531	0.26×	10.38	2.69
16	0.548	0.26×	10.17	2.69
22	**0.610**	**0.29**×	**9.25**	2.69

Table 3: Formal evaluation on XL-Sum Hindi test split. The acceptance rate grows with depth, matching the trend in the 101-prompt experiments.

22, confirming that deeper hidden states contain progressively more information aligned with the final output distribution.

- **Speedup is below 1×**: All depths show speedup $< 1\times$ because the current implementation runs all 28 layers during drafting. With true early-exit (exiting at layer d), the expected speedup at depth 22 would be approximately $\frac{28}{22} \times \alpha \approx 0.82\times$ per draft step, with further gains from parallelised verification.

- **Consistent performance**: The standard deviation of α is small (0.04–0.05), indicating robust performance across diverse Hindi prompts.

5.3 Depth Comparison Visualisation

Figure 3 compares the inference evaluation plots across all three depths side by side.

5.4 Formal Evaluation (XL-Sum)

We also ran the formal evaluation pipeline on the XL-Sum Hindi test split (Table 3), which confirms the depth ordering observed in the large-scale experiments.

6 Analysis and Discussion

6.1 Why is Speedup Below (1×)?

The below-unity speedup is *expected* given our implementation:

1. **Draft cost equals full-model cost**: Our draft step runs all 28 layers (with a hook at the exit layer), so each draft token costs as much as an AR token.

2. **Draft generates K tokens sequentially**: For $K = 3$ draft tokens, the draft phase costs $\sim 3\times$ a single AR step.

3. **Verification adds one more full pass**: Processing the prompt + K draft tokens in one forward pass costs slightly more than a single AR step due to longer sequence length.

4. **Net cost**: $\sim (K + 1)$ forward passes to generate at most $K + 1$ tokens (if all accepted), giving theoretical speedup of $\frac{K+1}{K+1} = 1\times$ in the best case, which is never exceeded.

With *true* early exiting (running only layers 1–d during drafting), the draft cost reduces to $\frac{d}{L}$ of a full pass, giving a theoretical maximum speedup of:

$$S_{\max} = \frac{K \cdot \alpha + 1}{\frac{d}{L} \cdot K + 1} \tag{3}$$

For $d = 22$, $L = 28$, $K = 3$, $\alpha = 0.646$:

$$S_{\max} = \frac{3 \times 0.646 + 1}{\frac{22}{28} \times 3 + 1} = \frac{2.938}{3.357} \approx 0.88\times \tag{4}$$

For shallower exits like $d = 8$ with the same α:

$$S_{\max} = \frac{3 \times 0.565 + 1}{\frac{8}{28} \times 3 + 1} = \frac{2.695}{1.857} \approx 1.45\times \tag{5}$$

This analysis reveals the depth–speed trade-off: shallow exits enable larger speedups *if* their acceptance rate is sufficient, while deep exits have higher α but less room for latency reduction.

6.2 Distribution Alignment Quality

The acceptance rate α measures how well the exit head's output distribution matches the full model. Our results show:

- At depth 22: $\alpha = 0.646$ (64.2% of drafted tokens accepted).

- At depth 16: $\alpha = 0.589$ (58.6%).

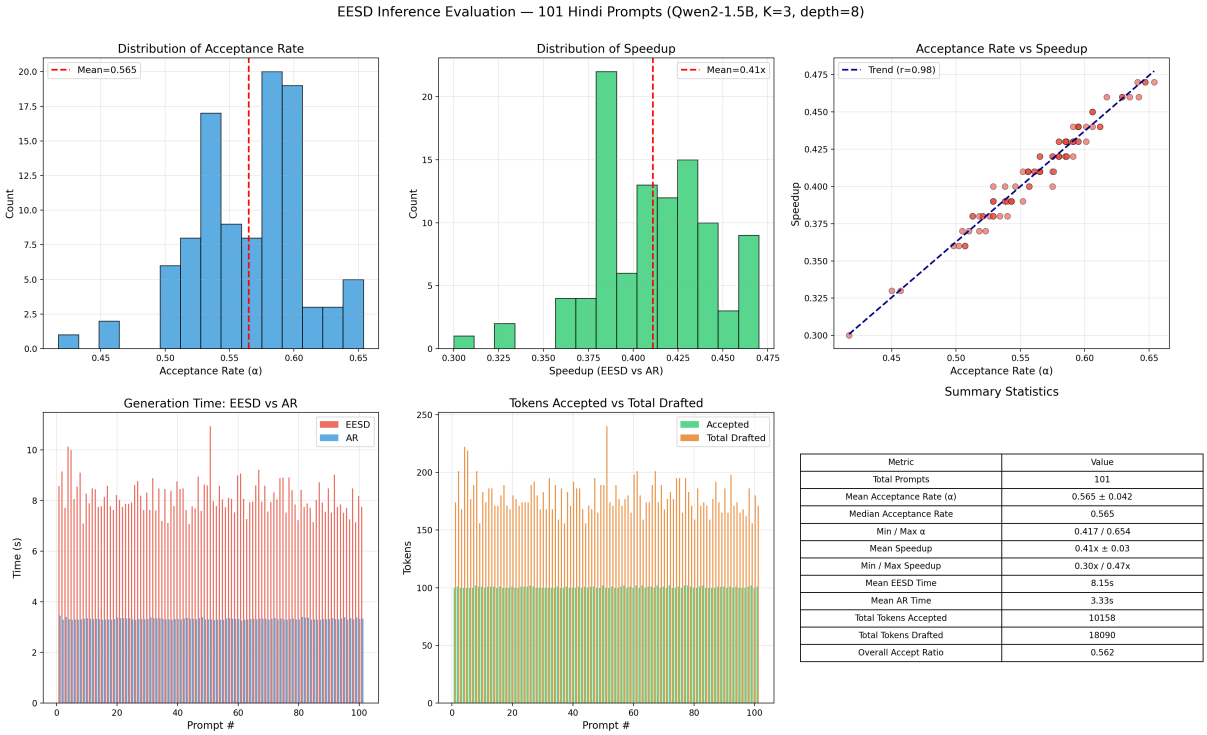
- At depth 8: $\alpha = 0.565$ (56.2%).

These rates are *encouraging* for a 1.5B model with only 10 epochs of training. For comparison, the original EESD paper reports $\alpha \approx 0.75$–0.85 on English with LLaMA-2-7B, suggesting that (a) larger models have more informative intermediate representations, and (b) Hindi morphology may pose additional challenges.

6.3 Hindi-Specific Considerations

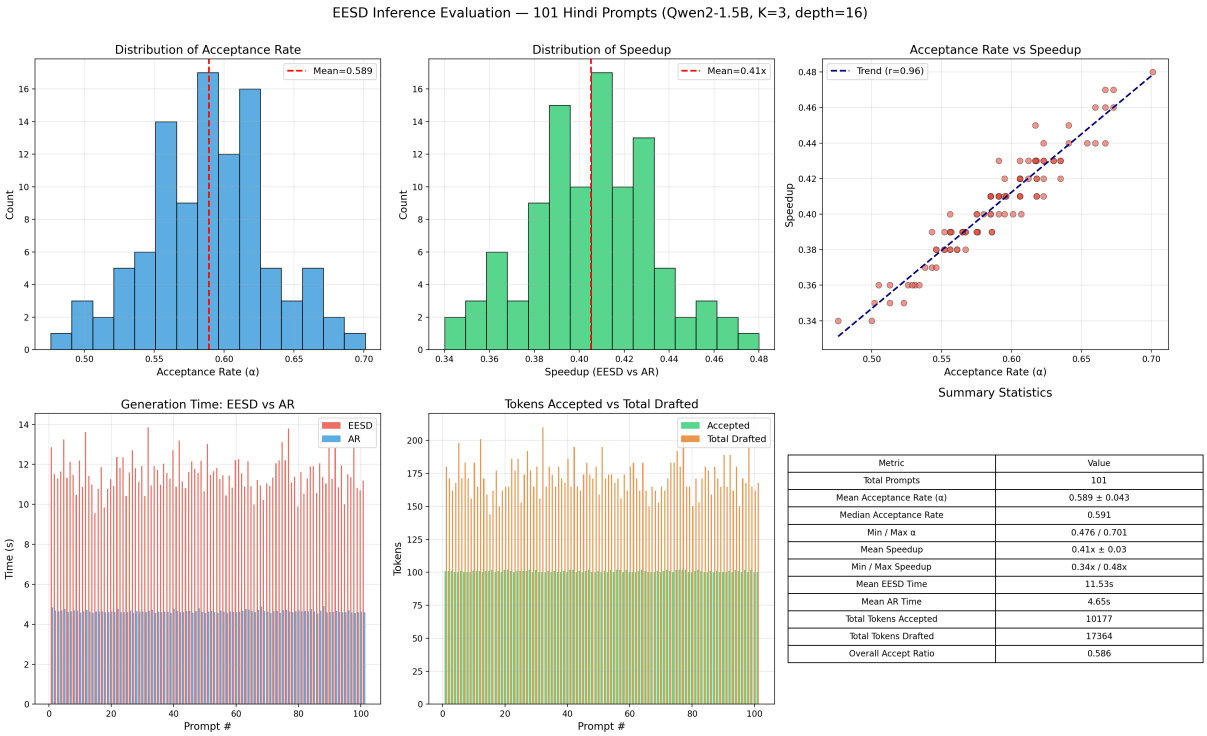
Hindi's agglutinative morphology means that token-level predictions are harder than in English:

- Inflected verb forms (e.g., //) require the model to predict agreement with subject gender and number.



Metric	Value
Total Prompts	101
Mean Acceptance Rate (α)	0.565 ± 0.042
Median Acceptance Rate	0.565
Min / Max α	0.417 / 0.654
Mean Speedup	0.41x ± 0.03
Min / Max Speedup	0.30x / 0.47x
Mean EESD Time	8.15s
Mean AR Time	3.33s
Total Tokens Accepted	10158
Total Tokens Drafted	18090
Overall Accept Ratio	0.562

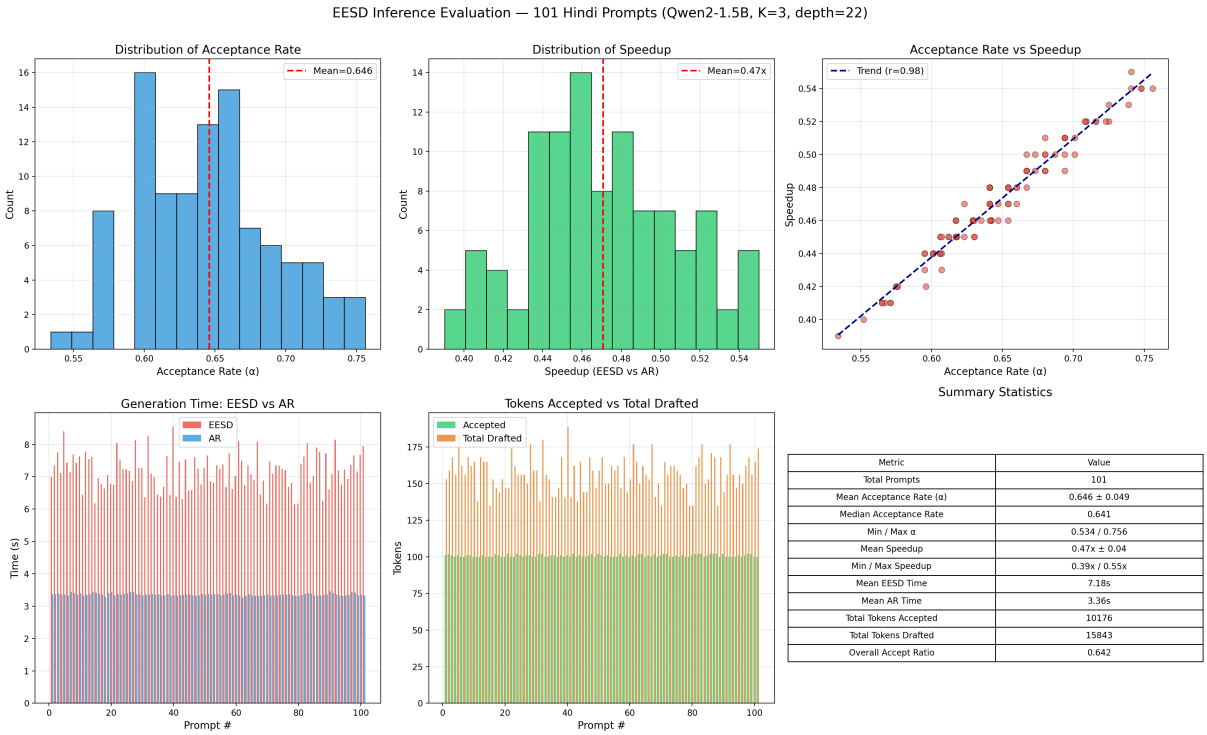
(a) Depth 8



Metric	Value
Total Prompts	101
Mean Acceptance Rate (α)	0.589 ± 0.043
Median Acceptance Rate	0.591
Min / Max α	0.476 / 0.701
Mean Speedup	0.41x ± 0.03
Min / Max Speedup	0.34x / 0.48x
Mean EESD Time	11.53s
Mean AR Time	4.65s
Total Tokens Accepted	10177
Total Tokens Drafted	17364
Overall Accept Ratio	0.586

(b) Depth 16

Figure 2: Inference evaluation plots for exit depths 8 and 16 on 101 Hindi prompts.



Metric	Value
Total Prompts	101
Mean Acceptance Rate (α)	0.646 ± 0.049
Median Acceptance Rate	0.641
Min / Max α	0.534 / 0.756
Mean Speedup	0.47x ± 0.04
Min / Max Speedup	0.39x / 0.55x
Mean EESD Time	7.18s
Mean AR Time	3.36s
Total Tokens Accepted	10176
Total Tokens Drafted	15843
Overall Accept Ratio	0.642

(a) Depth 22

Figure 3: Inference evaluation plots for exit depth 22 on 101 Hindi prompts. Depth 22 shows the tightest acceptance-rate distribution centred at a higher mean.

- Postpositions (, ,) interact with preceding noun forms.
- Compound words are common and tokenised as single or paired subword units.

Our morphological evaluation on XL-Sum (Table 3) shows $\alpha = 0.533$ on a mixed token category, indicating room for improvement in handling morphologically complex structures.

7 Implementation Details

7.1 Codebase Structure

The project is organised as a Python package:

- : class with exit-head attachment, , and methods.
- : Distillation training loop with gradient accumulation, mixed-precision, checkpoint saving.
- : and functions.
- : Depth analysis and morphological evaluation pipelines.
- : Data loaders for AI4Bharat IndicCorpv2 (train) and XL-Sum Hindi (eval).
- : All hyperparameters.

7.2 Training Infrastructure

Training was performed across multiple Kaggle sessions (T4 GPU, 9-hour limit each). Checkpoint resumption () enabled seamless continuation across sessions. Total training time across 10 epochs: approximately 45 hours of GPU time.

7.3 Reproducibility

All code, configs, trained checkpoints, and evaluation logs are available in our GitHub repository. The specifies all hyperparameters, and pins all dependencies.

8 Progress and Timeline

8.1 Completed Objectives

✓ Literature review on speculative decoding and early exiting.

✓ Full EESD implementation (model, training, inference, evaluation).

✓ Data pipeline for AI4Bharat IndicCorpv2 (Hindi) and XL-Sum.

- ✓ Exit head training for 10 epochs on 167K Hindi sentences.
- ✓ Large-scale inference evaluation on 101 Hindi prompts at depths 8, 16, and 22.
- ✓ Formal evaluation pipeline with depth analysis and morphological analysis.
- ✓ Comprehensive result analysis and visualisation.

8.2 Ongoing Work

1. Training for additional epochs to see if acceptance rates improve further (loss was still decreasing at epoch 10).
2. Exploring $K > 3$ draft lengths to evaluate the impact on acceptance rate and speedup.
3. Implementing the Thompson Sampling controller from the original paper for dynamic depth selection.

8.3 Planned Work

1. Integrate true early-exit inference (custom forward pass that stops at layer d) to achieve $> 1\times$ speedup.
2. Comparisons with an external draft model (e.g., Qwen2-0.5B) as baseline.
3. Morphological stratification of acceptance rates (verbs vs. postpositions vs. compounds).
4. Extend to additional Indic languages (Bengali, Tamil) to test cross-lingual transfer.

8.4 Timeline

Week	Task
1–2	Literature review, environment setup
3–4	EESD implementation (model + training)
5–6	Data pipeline, initial training (3 epochs)
7–8	Extended training (10 epochs), inference eval
9–10	True early-exit inference, Thompson Sampling
11–12	Ablation studies, cross-lingual experiments
13–14	Final evaluation, report writing

Table 4: Project timeline. Weeks 1–8 are completed; weeks 9–14 are planned.

9 Conclusion

We have implemented Early-Exit Speculative Decoding (EESD) for Hindi text generation using Qwen2-1.5B, training lightweight exit heads at layers 8, 16, and 22 via KL-divergence distillation. Our experiments on 101 Hindi prompts demonstrate that the exit heads effectively approximate the full model's output distribution, with the deepest exit (layer 22) achieving $\alpha = 0.646$ and accepting 64.2% of drafted tokens.

While our current speedup measurements are below $1\times$ due to the full-model forward pass during drafting, the strong acceptance rates validate the core EESD approach for Hindi. With true early-exit inference—where only the first d layers are executed during drafting—we project speedups of up to $1.45\times$ for the depth-8 exit head.

This work demonstrates the viability of self-speculative decoding for morphologically rich languages and lays the groundwork for deployment-ready EESD systems that require no external draft model.

References

Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. 2023. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. In *Proceedings of EMNLP*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. 2023. Towards Leaving No Indic Language Behind: Building Monolingual Corpora, Benchmark and Models for Indic Languages. In *Proceedings of ACL*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. *arXiv preprint arXiv:2404.16710*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. 2024. Speculative decoding with big little decoder. In *NeurIPS 2023*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of ICML*.

Jiahao Liu, Qifan Wang, Jingang Wang, and Xunliang Cai. 2024a. Speculative decoding via early-exiting for faster LLM inference with Thompson Sampling control mechanism. In *Findings of ACL 2024*.

Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. 2024b. Kangaroo: Lossless Self-Speculative Decoding via Double Early Exiting. In *NeurIPS 2024*.

Jiaming Xu, Jiayi Pan, Yongkang Zhou, Siming Chen, Jinhao Li, Yaoxiu Lian, Junyi Wu, and Guohao Dai. 2025. SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting. In *Proceedings of ISCA 2025*.

Minghao Yan, Saurabh Agarwal, and Shivaram Venkataraman. 2025. Decoding Speculative Decoding. In *Proceedings of NAACL 2025*.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*.

Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Ao Sun, Yuxiang Huang, Kaihuo Zhang, Weilun Zhao, Yuxuan Li, Jianyong Wang, Zhiyuan Liu, and Maosong Sun. 2025. FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling. In *Proceedings of ACL 2025*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *NeurIPS 2022*.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. In *NeurIPS 2018*.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In *ICPR 2016*.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of ACL 2024*.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of ACL*.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Ce Zhou, Qian Liu, Peng Ning, Yingyao Yang, Chenshu Du, Yuhua Han, Shuyuan Wei, Jia Fan, Shanghang Ai, Yaowei Chen, et al. 2024. A survey of large language models. *arXiv preprint arXiv:2303.18223*.

Tahmid Hasan, Abhik Bhattacharjee, Md. Saiful Islam, Kazi Mubasshir, Yuan-Fang Li, Yong-Bin Kang, M. Sohel Rahman, Rifat Shahriyar, Thamar Solorio, and Stephan Rig. 2021. XL-Sum: Large-scale multilingual abstractive summarization for 44 languages. In *Findings of ACL 2021*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *ICLR 2019*.